

v0.1.0 – BETA

EGM

Easy Game Maker

Build 2D games like a React developer — typed, tested, and deployable to any platform from a single command.

TypeScript

WebGL2

10 Platforms

How the Idea Was Born

A childhood obsession with 2D games — and a professional
obsession with software quality.

A childhood obsession with 2D games

Mega Drive. SNES. Hours spent in worlds built from pixels — and a burning question: *"Can I build something like this?"*

Not just a game. A game with unit tests, E2E tests, and CI/CD — built the same way we build great software today.

Mega Drive

SNES

TypeScript

Vitest

Playwright



The Problem with 2D Engines

Four critical gaps that keep game development stuck in the past —
while the rest of software engineering moves forward.

PROBLEM 01

The Introspection Problem "The Canvas Black Hole"

Unlike the web, where Cypress sees the DOM, game engines render to a canvas of *pixels*. External tools see nothing but a static image.

Consequence: Teams build proprietary "backdoor" APIs just to let tests validate game state. Months of infrastructure for what should be free.

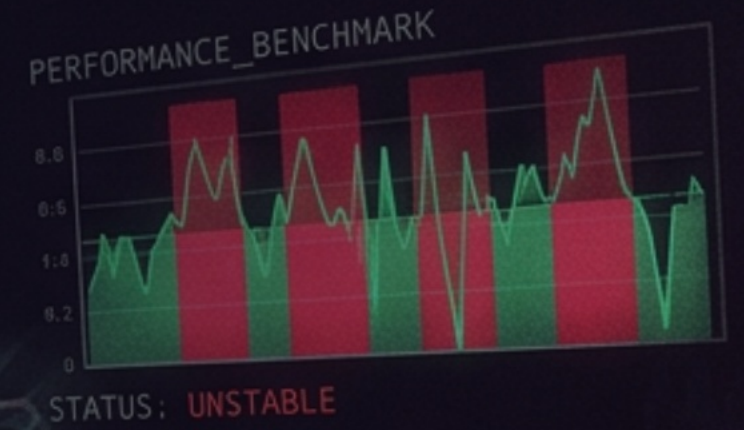


PROBLEM 02

Physics Instability Flaky Tests

Physics engines (Box2D, Chipmunk) are sensitive to FPS and floating-point precision. *Small hardware variations cause random test failures.*

Consequence: Automation loses trust. Teams return to slow, expensive manual testing cycles.



PROBLEM 03

CI/CD & Infrastructure Barriers

Engines depending on GUI and specific hardware make cloud automation a nightmare. Docker + GPU drivers + audio stacks on Linux = days of configuration.

Consequence: No DevOps culture. Slower releases. Manual QA gates that defeat the purpose of CI/CD.

PROBLEM 04

The Real Cost — The Financial Toll



Licensing

Pro/Enterprise tiers required for CI servers



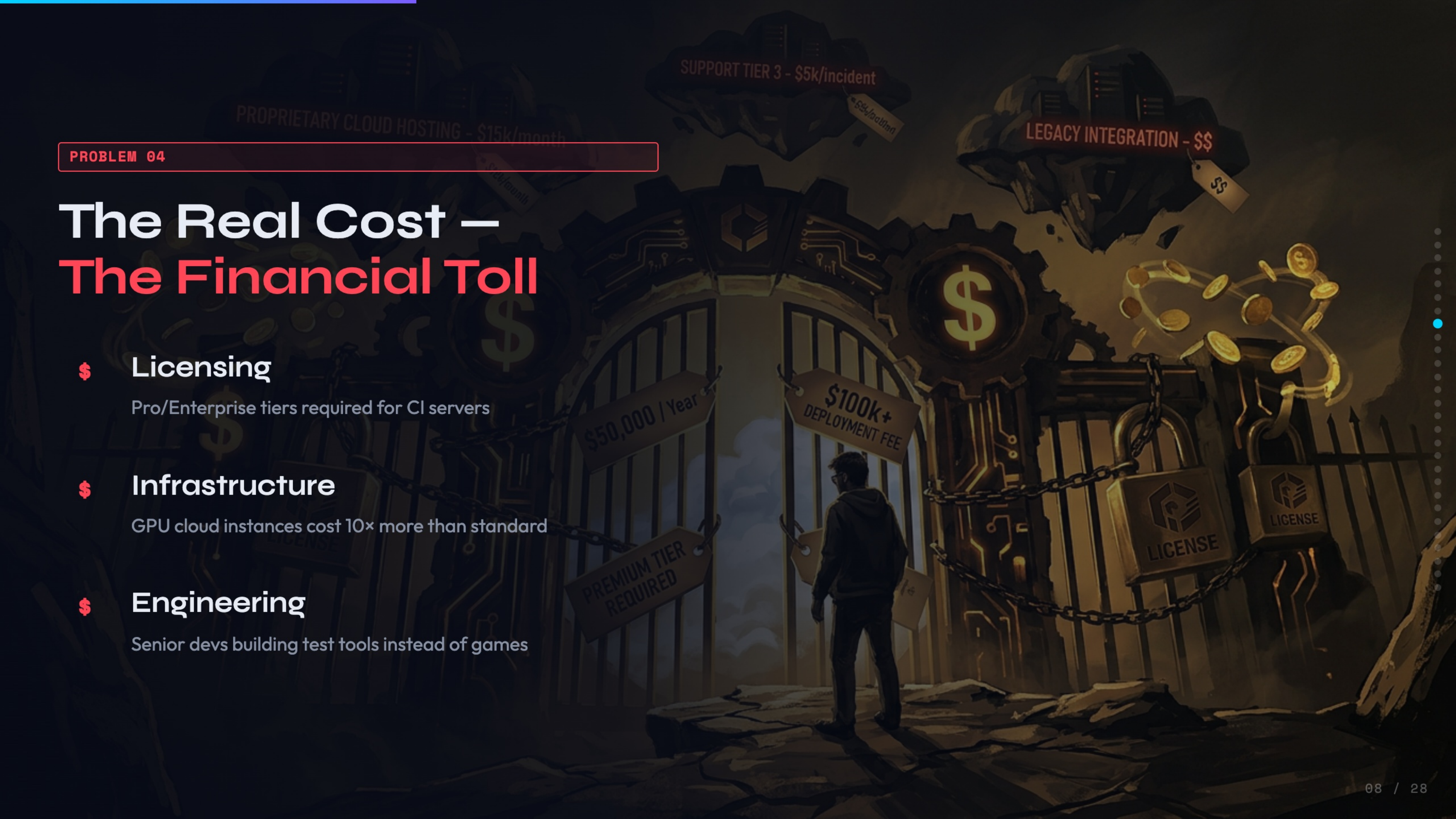
Infrastructure

GPU cloud instances cost 10× more than standard



Engineering

Senior devs building test tools instead of games



The Desired Outcome

What if building a 2D game felt exactly like building a modern React application — with the same developer experience, testing philosophy, and quality standards?

① WHITE-BOX TESTING

Game logic as testable as React

No backdoors, Mock + Scene Headless, no GPU.

② DETERMINISTIC PHYSICS

Fixed $dt=1/60$ — no flaky tests

planck.js TypeScript-native physics. Same result on every machine and CI agent.

Physics Bodies
colliders, rigid bodies,
constraints



③ ZERO-INFRA CI/CD

Any \$5/mo runner is enough

No GPU. No display servers. `[cli] test` + `[cli] e2e` on any Linux container.

④ ONE CODEBASE · 10 PLATFORMS

Write once. Ship anywhere.

Web, iOS, Android, Desktop, Smart TV, Consoles — *single* `[cli] build` *command*. No Pro licenses.

```
{ts} index.ts  
  
import ... from "cst";  
  
interface interface {  
  content {3};  
}  
  
function function() {  
  version = {}  
  return txpets;  
}
```

⑤ OUTSTANDING DX

The Real Differentiator

- **Simulator** — instant device preview + App to test in Smartphone
- **Visual Editor** — browser-based IDE with visual canvas + Monaco
- **Familiar Toolchain** — TypeScript, Vite, Vitest, Playwright
- **Single CLI** — new, simulate, editor, test, e2e, build

```
$ git status
/odventtevsare/developer
path "vitvanitos/come/adventure-git.txt"

$ npm run build-game
$ npm run build-game

$ deploy-to-simulator

$ tail -f logs/

... Build Successful. Simulator Updating...
```

"Done" means the same thing it does in modern web development — **typed, tested, visually editable**, and shippable to any platform from a **single command**.

THE EASY GAME MAKER PROMISE

EGM

The Beginning

Inspired by Solar2D, Unity 2D, and Godot — but uncompromising on software quality. Three critical decisions that changed everything.

Three Decisions That Defined Everything

TypeScript

Strongly typed. Web-native. Familiar.

Same language a React developer already knows. No barriers to entry.

WebGL2

The key to "deploy anywhere"

Runs in every modern WebView — iOS, Android, Desktop, Smart TVs, Consoles.

planck.js

TypeScript Box2D — deterministic

No WASM, no serialization. Fully testable physics from day one.

PROJECT: NEXT-GEN GAME ENGINE



DRAWN BY:
ARCHITECTS OF CODE

15 / 28

DEPLOY ANYWHERE STRATEGY

WebGL2 + Native WebViews

Every modern platform ships a WebView that supports WebGL2. The game is packaged as a standard web bundle — each platform wraps it in its native container.

📱 iOS — WKWebView

🤖 Android — Android WebView

💻 Desktop — Tauri 2.x (3-5MB)

📺 Tizen/WebOS — Web Runtime

🎮 PlayStation — PS WebView

🎮 Xbox — UWP WebView

WEBGL2
CORE

TRADITIONAL DEVELOPER

AI-ENHANCED DEVELOPER

THE AI MULTIPLIER

Development Time

TRADITIONAL

-6

months

Full engine — renderer, physics, audio, CLI, simulator, visual editor, and all 10 platform builders. Not just a timeline — proof of concept for the future of software development.

AI-ASSISTED

8 days

What EGM Provides

Two developer profiles. One complete engine. Everything needed to build, test, and ship a 2D game — in TypeScript.

CODE-FIRST

Two profiles. One engine.

CODE-FIRST

egm new <name>

Full TypeScript control. Write scenes, objects, logic — everything in code. Ideal for developers who want precision and maximum flexibility.

VISUAL EDITOR

VISUAL EDITOR

egm new <name> --visual

Scene layouts in `views/*.view.json` (edited visually).
Game logic in `events/*.events.ts`. Design visually, code behavior.

Everything a 2D game needs in TypeScript

Category	APIs
Display	Sprite, AnimatedSprite, RectShape, CircleShape, Text, ParticleEmitter...
Scene	Scene, SceneManager, VisualScene, ViewLoader
Physics	PhysicsWorld, PhysicsBody (planck.js Box2D)
Camera	follow, shake, pan, zoom, fade/flash
Input	Keyboard/Mouse/Touch + GamepadManager
Audio	AudioManager, AudioChannel (Web Audio API)
Network	NetworkManager, NetworkRoom (WebSocket)
Gameplay	ObjectPool, StateMachine, Tilemap (Tiled JSON)
Monetization	AdManager (AdMob), IAPManager (In-App Purchases)
Testing	MockApp, MockRenderer, SceneRunner — headless

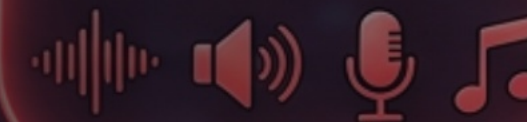
Display Objects



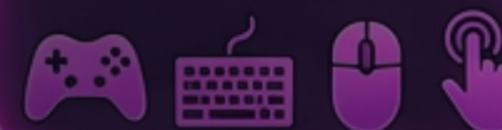
Physics



Audio



Input



Camera



Network



Testing



TS

EGM Editor

A browser-based IDE built for game development. No external tool required — but your existing tools still work.

`egm editor`

Code. Visualize. Ship.

CODE-ONLY MODE

File tree, Monaco editor (TypeScript/JSON/CSS), Ctrl+S → instant HMR, File CRUD with right-click

VISUAL MODE

Live canvas · Object Palette · Inspector · Scene CRUD · Undo/Redo (Ctrl+Z) · Events tab · Zoom

Execute — opens the full EGM Simulator in a new tab. Use any external editor (VSCode, Webstorm) — EGM provides its own built-in IDE for the best integrated experience.

Property Inspector

Player_Group

Transform

Position X 145 Y 280

Rotation 0

Scale X 1.0 Y 1.0

Color

Fill #FFDD44

Opacity 1.0

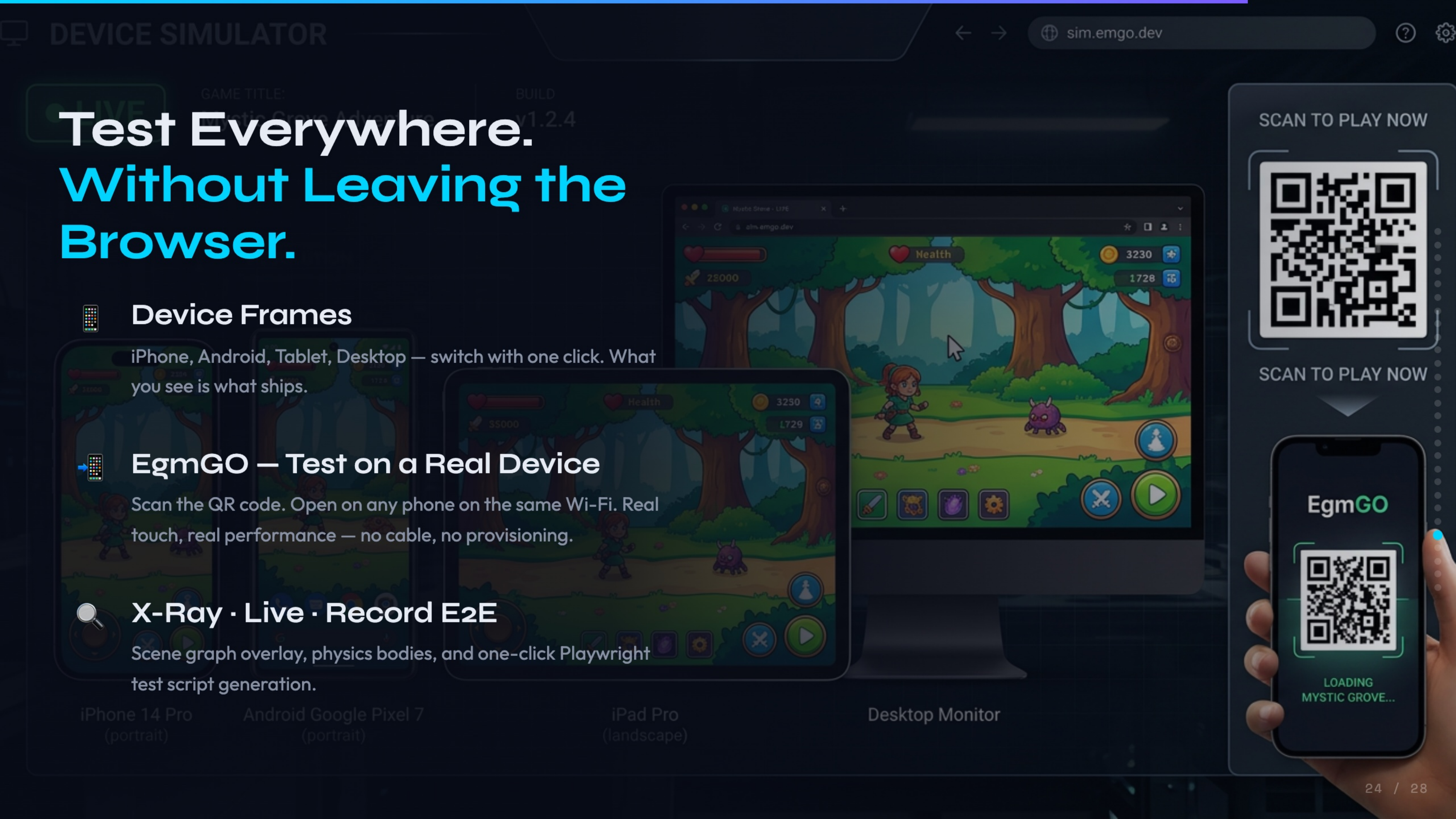
player.js ×

```
1 function update() {  
2   var var () {  
3     var Position = 145;  
4     player.width = 80;  
5     player.top = 280  
6   }  
7  
8   function physic() {  
9     var position = bot.sinelect  
10    player.all = nag.eotPesitic  
11    player.mot.length = this.st  
12  }  
13 }  
14  
15 function update(( )) {  
16
```

EGM Simulator

Fully web-based device simulator. No Xcode. No Android Studio. Just a browser — and a real device in your hand.

```
egm simulate
```



Test Everywhere. Without Leaving the Browser.

Device Frames

iPhone, Android, Tablet, Desktop — switch with one click. What you see is what ships.

EgmGO — Test on a Real Device

Scan the QR code. Open on any phone on the same Wi-Fi. Real touch, real performance — no cable, no provisioning.

X-Ray · Live · Record E2E

Scene graph overlay, physics bodies, and one-click Playwright test script generation.

iPhone 14 Pro
(portrait)

Android Google Pixel 7
(portrait)

iPad Pro
(landscape)

Desktop Monitor

SCAN TO PLAY NOW



SCAN TO PLAY NOW

EgmGO



LOADING
MYSTIC GROVE...

EGM Build

One TypeScript codebase. One command. Ten platforms. No rewrites.
No licenses. No surprises.

```
egm build <platform>
```

EPIC PRODUCT LAUNCH

UNIFIED BUILD DEPLOYMENT

10 Build Targets.
Zero Rewrites.

CLI: BUILD & LAUNCH PIPELINE

```
$ BUILD_TARGETS=10  
STATUS: SUCCEEDED  
DEVICES: 10/10  
LAUNCHING GAME...  
TOTAL_BUILD_TIME: 2.1s  
GAME_TITLE: STARFORGE ADV.  
SUCCESS: ALL CHANNELS ACTIVE
```

UNIFIED BUILD DEPLOYMENT

{TS}

TypeScript Bundle



Web



iOS



Android



Desktop (macOS/Win/Linux)



Samsung Tizen



LG WebOS



Android TV



tvOS (Apple TV)



PlayStation



Xbox



BUILD SUCCESSFUL



BUILD SUCCESSFUL



BUILD SUCCESSFUL



BUILD SUCCESSFUL



BUILD SUCCESSFUL

iPhone

iPhone

Android

Windows PC

MacBook

Windows PC

Android TV box

PlayStation

PS5 console

Xbox Series X

The developer never leaves TypeScript

```
$ egm build web          # → dist/web/
$ egm build ios          # → dist/ios/ (Xcode)
$ egm build android      # → dist/android/ (Gradle)
$ egm build desktop      # → dist/desktop/ (Tauri)
$ egm build tizen        # → dist/tizen/ (Samsung TV)
$ egm build xbox         # → dist/xbox/ (UWP)
```

The toolchain handles the rest. Same TypeScript. Same tests. Every platform.

✓ Build Finished! (All platforms 100% green)

dev@devmachine:~\$

100%
Path: /builds/web/prod/v3.1.5/dist/

100%
Path: /builds/ios/release/v3.1.5/MyGame.ipa

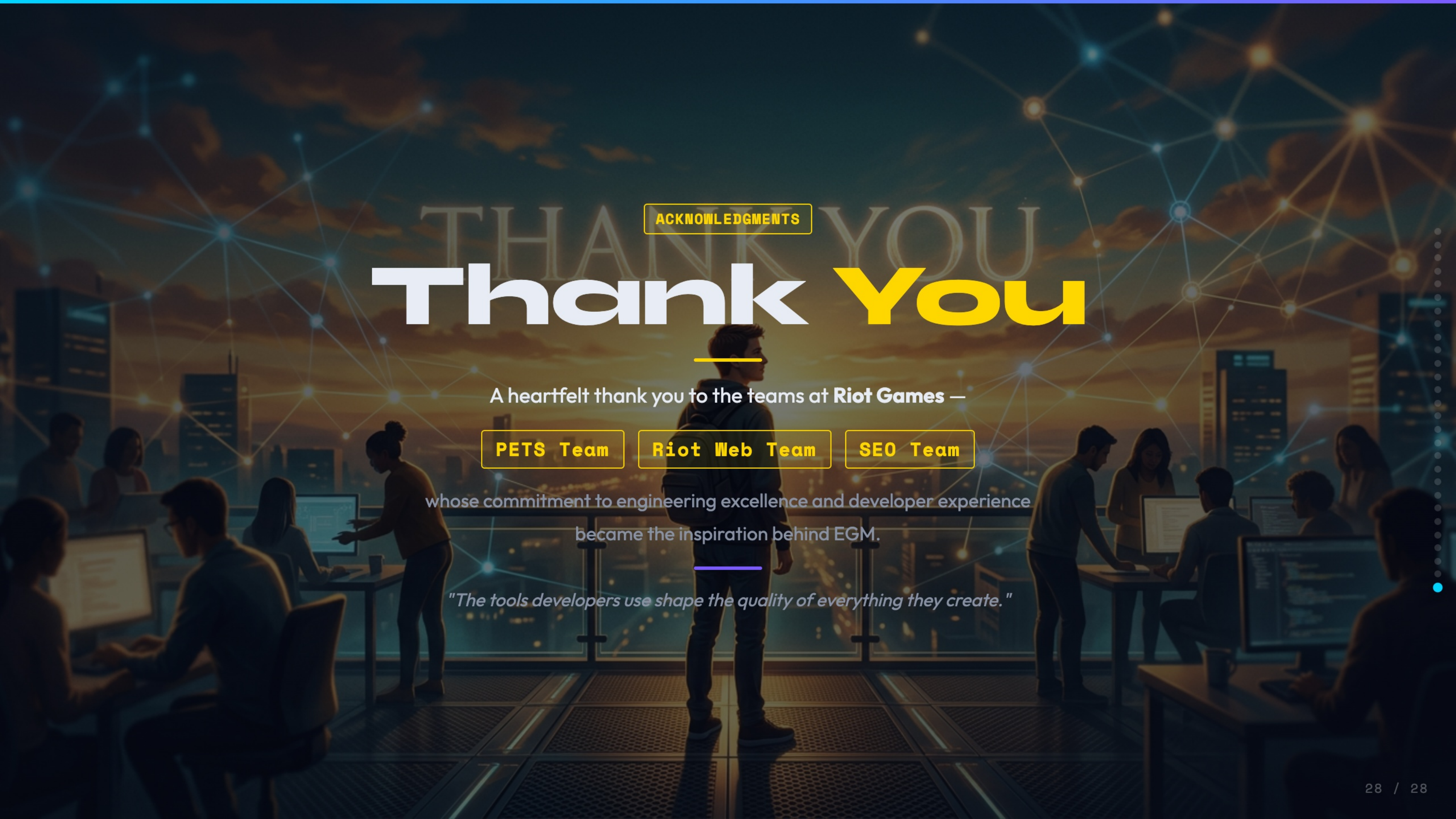
100%
Path: /builds/android/prod/v3.1.5/MyGame.apk

100%
Path: /builds/desktop/dist/v3.1.5/MyGame_Win_mac_Lin/

100%
Path: /builds/tizen/prod/v3.1.5/MyGame.wgt

100%
Path: /builds/xbox/dev/v3.1.5/MyGame_Xbox.appx

✓ 14:32:01



THANK YOU Thank You

ACKNOWLEDGMENTS

A heartfelt thank you to the teams at Riot Games —

PETS Team

Riot Web Team

SEO Team

whose commitment to engineering excellence and developer experience
became the inspiration behind EGM.

"The tools developers use shape the quality of everything they create."